Combining Pattern Recognition and Game Search for an Intelligent Tic-Tac-Toe Agent

Jeremy Qiao

CS4795 Introduction to Artificial Intelligence
University of New Brunswick
Student ID: 3753194

Abstract. For this project, I will create a Tic Tac Toe Agent, it will use pattern recognition and search algorithms to find the best move (action) from all available options. An intelligent agent is a software program, it can find a plan of actions which maximize the objective, it can change the plan by observing the environment and automatically execute the plan. An intelligent agent in playing games will analyze different possible states of a game and decide the best way to win.

Keywords: Artificial Intelligence · Search Algorithms · Pattern Recognition · Tic-Tac-Toe

1 Introduction

For this project, I will create a Tic Tac Toe Agent, it will use pattern recognition and search algorithms to find the best move (action) from all available options.

An intelligent agent is a software program, it can find a plan of actions which maximize the objective, it can change the plan by observing the environment and automatically execute the plan. An intelligent agent in playing games will analyze different possible states of a game and decide the best way to win.

I will represent the Tic-Tac-Toe board as a state space, where each board position relates to a state and any given move creates a new state from the previous one. It uses a search algorithm to search through all the possible states on the Tic-Tac-Toe board and find which move will result in the best overall result for that agent.

2 System Design

The system will include two main components:

2.1 Pattern Recognition

Pattern recognition will analyze the board to detect important patterns:

- Winning pattern (three in a row)
- Threat pattern (two opponent marks in a row)
- Strategic positions (center and corners)

These patterns will help evaluate the current board state and guide the search process.

2.2 Game Search Algorithms

I plan to use different search algorithms to explore possible game states, including:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Depth-Limited Search
- Iterative Deepening Search

These algorithms explore the state space by expanding nodes in different orders. The search strategy decides which node will be expanded next. Different strategies have different time complexity and space complexity, and they may or may not be complete, and may or may not be optimal for this situation.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	b^m	b^l	b^d
Space	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	bm	bl	bd
Optimal?	Yes*	Yes	No	No	Yes*

Fig. 1. Comparison of search algorithms including completeness, time complexity, space complexity and optimality.

3 Evaluation Metrics

So, the performance of each algorithm will be evaluated by measuring:

- completeness — the four algorithms will be tested to see if they can always find an optimal move in the game.
- time complexity — it will be evaluated by measuring how many game nodes (like different states) each algorithm expands before choosing a move.
- space complexity — it will be measured by the maximum number of game states stored in memory during the search.
- optimality — it will check if the algorithm always selects the best possible move

The goal of this comparison is to understand how different search strategies affect decision-making in game-playing agents.

The final system will allow the AI agent to play Tic-Tac-Toe against:

- a random player
- another AI agent

Experimental results will show which search algorithm performs best when selecting the optimal move.

4 Agent and Problem Formulation

So overall, my Tic-Tac-Toe Agent is a deterministic, fully observable problem (single-state problem). Because the agent can see the entire board and the result of placing a mark is predictable. Additionally, the next state of the game is fully determined by the move. It is also a Goal-based agent (problem-solving agent). It searches through possible game state and selects actions that lead to a goal.

Performance: Win the game or lose the game

Environment: The Tic-Tac-Toe board (3×3 grid). (Maybe I will increase the size I am not too sure right now)

Actuator: Place a mark (X or O) in an empty cell.

Sensor: Observe the current board state.

States: All possible Tic-Tac-Toe board configurations.

Actions: Place a mark in any empty cell on the board.

Goal Test: Reaches a goal if the agent has three marks in a row or the game ends in a draw.

Path Cost: Each move costs 1 step.

5 Project Timeline

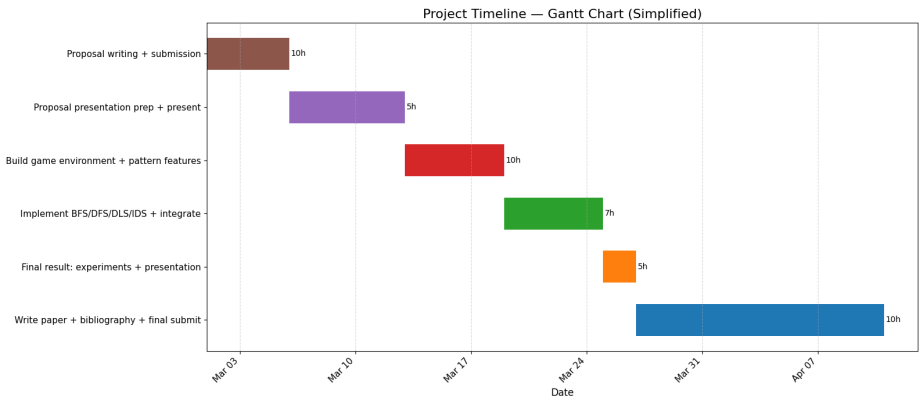

Fig. 2. Project timeline for the Tic-Tac-Toe agent development and evaluation.

6 Project Repository

GitHub Link:

`https://github.com/Jeremyy4/CS4795_AI_Tic-Tac-Toe-Agent`

7 Bibliography

References

1. Kim, J.-K.: Introduction to Artificial Intelligence. University of New Brunswick (2026)
2. Kim, J.-K.: Problem-Solving Agents and Search Algorithms. University of New Brunswick (2026)
3. Kim, J.-K.: Tree Search Algorithms and Uninformed Search. University of New Brunswick (2026)
4. Kim, J.-K.: Depth-Limited Search and Iterative Deepening Search. University of New Brunswick (2026)
5. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. Prentice-Hall (2010)
6. Bishop, C.: Pattern Recognition and Machine Learning. Springer (2006)